The Cache Collectors: Automatic Discovery of High-Performance Cache Replacement Policies

Meseker Worku, Ethan Goodman, Madison Torres, Savanah Hendzel,
Rithik Kulkarni, Brantley Miller
Department of Computer Science
North Carolina State University
Raleigh, North Carolina, USA

GitHub Repository Link

***Abstract*—This paper presents the Cache Collectors' work towards the CacheForge cache replacement policy generation competition. The challenge required using only LLMs to synthesize ChampSim-compatible replacement policies that outperform state-of-the-art baselines under a strict hardware constraint. We investigated methods of prompt engineering, evolutionary refinement, surrogate reward modeling, and agent-based decomposition to improve policy quality and system robustness.**

Our experimentation found that broad exploration proved ineffective in comparison to its counterpart, exploitation. Our flagship version enforced strict code constraints, required atomic mutations, and exploited previously generated top-performing strategies. This strategy enabled an open-source 20B-param model to surpass the 0.413081 IPC Hawkeye baseline, making us the first team in the competition to achieve this.

We also report negative findings. Feeding the LLM with micro-architecture context, explanations of policies, seeding with baseline policies, and even developing a brand-new multi-agent system architecture all demonstrated little assistance in increasing harmonic mean IPC. In contrast, exploit-only refinement paired with a surrogate filter proved far more effective. Our highest performing policies converged towards a common theme of spatial-aware frequency modulation.

Our results show that model size alone is not the primary determinant of success in this competition. Prompt discipline, proper constraint handling, and honing in on previous success were the actions that drove our performance considerably.

1 Introduction

Modern processors rely heavily on cache hierarchies to decrease the gap in latency between fast CPU cores and comparatively slower main memory. The cache stores frequently used data so the CPU can retrieve it quickly. When the cache is full, a replacement policy must decide which data to evict. Due to its complexity, cache replacement remains a critical optimization problem in computer architecture.

Recent advancements in artificial intelligence, especially Large Language Models (LLMs), have created new opportunities for automating cache policy design. Instead of manually crafting replacement policies, LLMs can generate candidate C++ policies that can be compiled and tested using architectural simulators. In this project, we use the CacheForge framework, which connects an LLM to the ChampSim evaluator to automatically generate, test, and evaluate cache replacement policies.

In our previous individual work with CacheForge, the framework operated as a stateless generate–compile–simulate loop. The LLM proposed new policies, which were evaluated across five workloads, and the system selected better policies based on the policy's average raw cache hit rate. This approach relied heavily on brute-force exploration, stored no memory of past design choices, and had little prompt optimization. As a result, ineffective ideas were often repeated, causing the exploration process to be inefficient and narrowly focused.

This collaborative work significantly expands this framework by transforming CacheForge into an experience-driven optimization system. Policies are now evaluated across ten workloads instead of five, expanding the focus to generalization and robust performance. Our goal was to generate a policy with an average IPC (Instructions Per Cycle) higher than

the several reference baseline policies while staying within a strict 64 KiB hardware storage limit.

2 Background

2.1 Cache Replacement

The real magic of caching is not just in the hardware's capacity, but in which blocks of memory are evicted from or retained in the cache. This is where the cache replacement policy comes into play. Cache replacement is a fundamental design challenge in computer architecture, as the processor must try to predict and evict the blocks of memory that will be reused farthest in the future. Despite this task seeming simple in explanation, accurately estimating the usefulness of a block is incredibly difficult in practice.

A major contributor to the complexity of this problem is the wide range of temporal and spatial access behaviors across various workloads. While some workloads demonstrate strong locality where certain blocks get repeatedly reused in a short amount of time, others follow patterns that rarely ever revisit blocks of memory at all. The theoretically optimal replacement policy will depend on recognizing these sorts of patterns ahead of time, which means systems must be able to detect predictive signals from a limited amount of data. While heuristics such as frequency or recency exist, they can only capture a fraction of this behavior. For example, reuse distance, which is the number of accesses between the reuses of a block, can vary widely across different workloads. In a similar situation, a frequency-based indicator may cause a policy to retain a few short-lived accesses thinking they are important, causing other more valuable blocks to be evicted early.

Some aspects of cache replacement that further complicate the matter include microarchitectural effects such as prefetching. The idea behind prefetching is to predictively retrieve data into the cache before it is actually requested. While prefetchers may improve latency, they also run the risk of evicting blocks that would have been reused soon. Additionally, the behavior of a system can also be impacted by how critical each access is to the overall system performance. Some cache hits matter more than others due to varying pipeline timing, data dependency, or whether an access lies on a critical path of execution for the processor.

These variances matter because the quality of a *policy* is often gauged by hit rate, despite hit rate not being the best measure of a *processor's* performance. Hit rate is defined as the fraction of accesses that are served by the cache instead of by main memory (Equation 1).

$$\text{Cache hit rate} = \frac{\text{\# of cache hits}}{\text{Total \# of cache accesses}} \quad (1)$$

Performance of a processor is quantified using Instructions Per Cycle (IPC), which measures how many instructions the processor retires per clock cycle. Due to the varying importance of cache hits, however, two policies with identical hit rates may demonstrate different IPC values because not all hits carry an equal processor performance boost.

For our previous work with CacheForge, we focused on increasing hit rate. In this work, we are focused on increasing IPC, as it takes into account the microarchitectural consequences of policy behavior, while hit rate merely counts raw cache hits. The true goal of a policy is to optimize IPC, not hit rate.

At a theoretical level, an optimal cache replacement policy is represented by Belady's optimal policy, which simply evicts the block that is farthest from reuse. Although this is quite obvious, it is nearly impossible to reliably predict the least useful block consistently. Instead, Belady's optimal policy serves its role as an upper bound to motivate strategies that look to base eviction on local reuse history.

2.2 Reference Baselines

Past research has produced some popular heuristics for replacement including:

1) *Least Recently Used (LRU)*, which uses recency to predict reuse patterns
2) *Signature-based Hit Prediction (SHiP)*, which predicts reuse likelihood based on program context signatures
3) *Reordering*, which adjusts priority dynamically rather than just following recency

Policy	Cache Hit-Rate (HR)	Load HR	RFO HR	Prefetch HR	Writeback HR	IPC	Storage (KB)
Hawkeye	0.3369	0.2956	0.2828	0.2058	0.6969	0.4131	31.8
Multipersp.	0.2512	0.2176	0.2905	0.1496	0.4544	0.4129	29.75
Ship++	0.2965	0.2977	0.2574	0.1899	0.6176	0.4120	20.0
Less is More	0.2662	0.2340	0.3310	0.1049	0.5322	0.4096	31.2
Reordering	0.3235	0.2994	0.2551	0.2271	0.6864	0.4068	31.88
LRU	0.4022	0.3081	0.2469	0.2115	0.9998	0.4016	16.0

TABLE I: Reference Baselines

4) *Multiperspective*, which combines multiple signals into a unified replacement score
5) *Less fis More*, which emphasizes certain minimal features over broader predictors
6) *Hawkeye*, which attempts to approximate Belady's by learning which past blocks would have been reused

These heuristics are each associated with one of the six baseline policies we aimed to surpass in IPC. While these baselines cover many different philosophies that even sometimes conflict, it is important to note that no single baseline dominates across all workloads; furthermore, a policy like LRU achieves the highest average hit-rate but the lowest IPC (Table I). This serves as proof that optimizing IPC cannot be done simply by maximizing raw cache hit-rate. [3]

This helps us realize two key realities of this problem. First, that architectural performance is more complicated than hit-rate. Second, that heuristics are only effective on workloads that match their underlying assumptions.

Our evaluation of policies is based on a select 10 workloads. These workloads and their behavior can be compiled as the following:

- ***astar*** is a search algorithm that sometimes reuses its data, but other times moves on, making reuse unpredictable.
- ***calculix*** is an engineering simulation with regular computations and moderate reuse.
- ***gcc*** is a compiler where some data is reused, but the patterns frequently change.
- ***hmmer*** is pattern-matching where short bursts of reuse are followed by long streams of new data.
- ***lbm*** is a fluid simulation which has very little reuse and therefore causes many misses.
- ***mcf*** is a graph solver with rare reuse.
- ***milc*** is a physics simulation that stresses prefetch accuracy.
- ***namd*** is molecular dynamics code that emphasizes identifying important long-term data.
- ***omnetpp*** is an event-driven simulator where behavior depends on the program's phase, forcing the policy to adapt.
- ***zeusmp*** is a scientific solver that uses the same region of memory frequently, but has long streaming regions.

Together, these baselines will help us by serving as anchors for our search of the cache replacement policy design space. [2]

2.3 Related Work

FunSearch serves as the inspiration for CacheForge's evolutionary design loop and prompting. Known also as "Searching in Function Space," FunSearch pairs a pretrained LLM with a systematic evaluator. This function produces programs iteratively rather than producing output directly like other computer search techniques. The system can iteratively generate, test, and refine these programs, helping them to become more interpretable and precise. FunSearch utilizes an 'evaluate' function to score outputs and decide which solutions to improve on. The program starts as a skeleton and only the critical logic is evolved. Over many iterations, low-scoring programs evolve by discovering new knowledge. These high-scoring programs are then reintroduced to the LLM to spark improvement, known as "best-shot prompting." FunSearch's effectiveness is seen through its ability to surpass best-known results in important problems, most notably the cap set problem. Its novelty is seen through its contrast to most computer search approaches. It is most effective when a problem is described well

rather than just given an example solution, which allows these programs to evolve themselves using the LLM as a generator and the evaluator as a filter. [1]

CacheForge directly adapts the original FunSearch architecture, treating cache replacement policies as mutable C++ programs in the evaluation context of ChampSim, similar to the programs that FunSearch generates. In Homework 1, the framework used was likened to a FunSearch-styled loop applied to microarchitectural design. In this, the LLM generated candidate replacement logic, the system compiled and simulated it, and high-performing results were fed back into the query prompts. Both of the systems relied on performance metrics for evolution, policies and programs being measured on quality, and gaining knowledge through iterations and previous policies rather than a single output. However, there is a big difference between the two in terms of evaluation. FunSearch works to evaluate solutions based on mathematical correctness and CacheForge works through ChampSim with IPC and hit-rate metrics. With these similarities and differences, the two connect with FunSearch as a predecessor that showed evolution through iterative evaluation. This enabled CacheForge to build on the same techniques, but in the field of computer micro-architecture. [1]

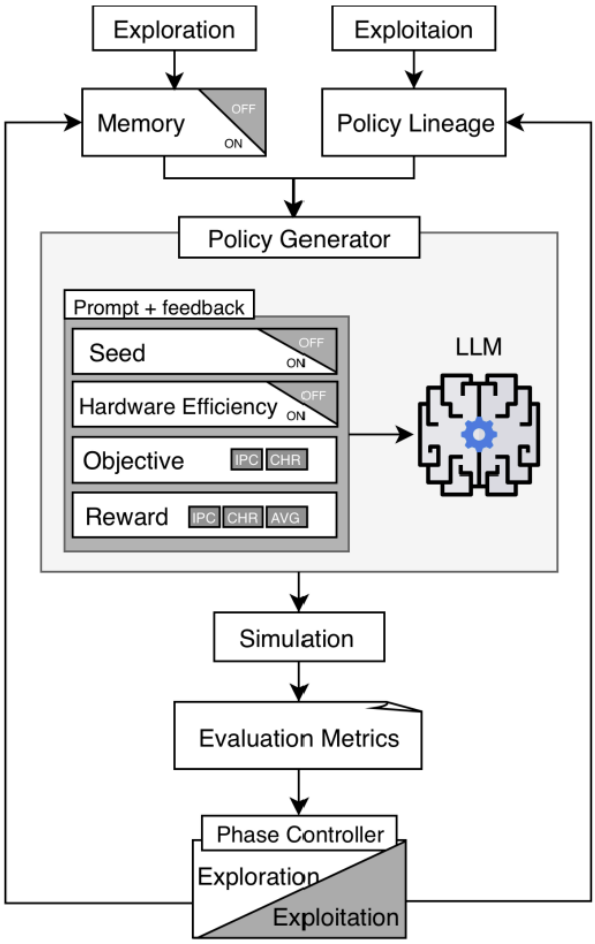

Fig. 1: CacheForge Pipeline

3 CacheForge

Unlike other traditional frameworks, CacheForge is the first framework that uses an LLM to explore the policy solution space while maintaining hardware constraints. CacheForge represents these policies as mutable C++ code, each evaluated by ChampSim with performance penalization. Policies are modifiable as loops, state machines, update rules, and tie-breakers are all components to be explored and evolved. This design extends beyond manually tuned replacement designs such as PACIPV and MPPPB, and allows the evaluator to perform the first instance of "autonomous design exploration for cache replacement" that goes past the limits of manual parameterizations.

ChampSim is a micro-architecture simulator designed for research, education, and competition in the field of computer architecture. As modern computer processors have grown increasingly complex, issues arise where fully comprehending the complexity of micro-architecture is difficult for students with little background knowledge. ChampSim addresses this with modular design and configurable structure such as branch predictors, cache replacement policies, and prefetchers. This allows modules to be tested quickly and effectively. ChampSim is trace-driven; workloads are simulated, returning results such as IPC and hit rate that serve as feedback on policies generated. It has been featured in many industry competitions including the Second Cache Replacement Championship and the Third Data Prefetching Championship, where innovative designs are evaluated by various metrics. ChampSim plays a central role in our project as the simulation engine responsible for evaluating cache replacement policies across different workloads, and is ideal for

CacheForge. [5]

The CacheForge pipeline uses a large generation loop that builds on an initial database of baseline policies. The LLM is given a code template and sufficient context to generate a C++ policy using two primary modes, *explore and exploit*. Explore mode prompts the LLM to explore the solution space by trying brand new replacement strategies, while exploit mode is enabled based on a temperature constraint. Exploit mode attempts to make small mutations to the existing policy in hopes of improving the performance while maintaining the already sound strategy. Every time a policy is generated and successfully compiles, it is evaluated across the 10 workloads mentioned earlier and appended to the initial database.

Additionally, as seen in the CacheForge System Design in Figure 1, CacheForge also has a memory system that keeps track of policies in both exploration and exploitation phases to feed the LLM with a recent history of generated policies and their performance. This enables the LLM to ignore strategies that have historically yielded poor performance and maintain bias towards those that yield high performance. [4]

4 Methodology

4.1 Evolution of CacheForge

Figure 2 visualizes the evolution of our CacheForge codebase throughout the entirety of the project. Each generation represents changes that were specifically built off of a version of the codebase from the previous generation.

4.2 Policy Explainer

The policy explainer was one of our earliest experiments, so our focus was less towards exploiting top performing policies and more towards enhancing LLM memory, consistency, and reliability with a policy explainer. To do this, every time the LLM was prompted to output a new policy, we included a `JSON_SCHEMA` that was required as output. This output included the following fields:

- `policy_name`
- `policy_description`
- `mutation_description`
- `cplusplus_implementation`
- `storage_kb_est`
- `storage_note`
- `data_structures_used`
- `comparison_to_known_policies`
- `mutation_diff`

After generation, the JSON key/value pairs are extracted and stored inside an episode. If the policy belonged to a lineage, the episode is assigned a `lineage_id`. If at any time a lineage block with that id is used for a future prompt, that episode is included.

As discussed earlier, this design serves two main purposes. Firstly, it enhances the memory encoded inside lineage blocks which is used by the LLM to reconstruct what was previously attempted across prior generations. Secondly, since the model (in exploit mode) was required to take inspiration from ancestors and exploit their designs, forcing detailed mutation information through `mutation_description` and `mutation_diff` intended to decrease the likelihood of inventing changes or drifting away from the intended lineage constraints.

4.3 Surrogate Reward Model

To further our exploration into methods that could speed up our policy generation and avoid policies that either do not compile or deliver low average IPC, we decided to employ a surrogate reward model to weed out policies that, at first glance, were essentially garbage. To accomplish this, we began training a surrogate reward model. This predictive model is trained on a running JSON memory file that collected metadata for all generated policies over multiple experiment runs. This metadata included the following features:

- Policy Description from LLM (hash)
- Mutation Description from LLM (hash)
- Parent Policy's IPC
- Estimated storage in KiB
- # of static array declarations in the C++ file
- # of for loops in the C++ file
- Length of the C++ code in characters

The goal of the model is to predict the ΔIPC from the parent policy's IPC. We train the model to predict ΔIPC rather than directly predicting an average IPC value because it provides a more stable

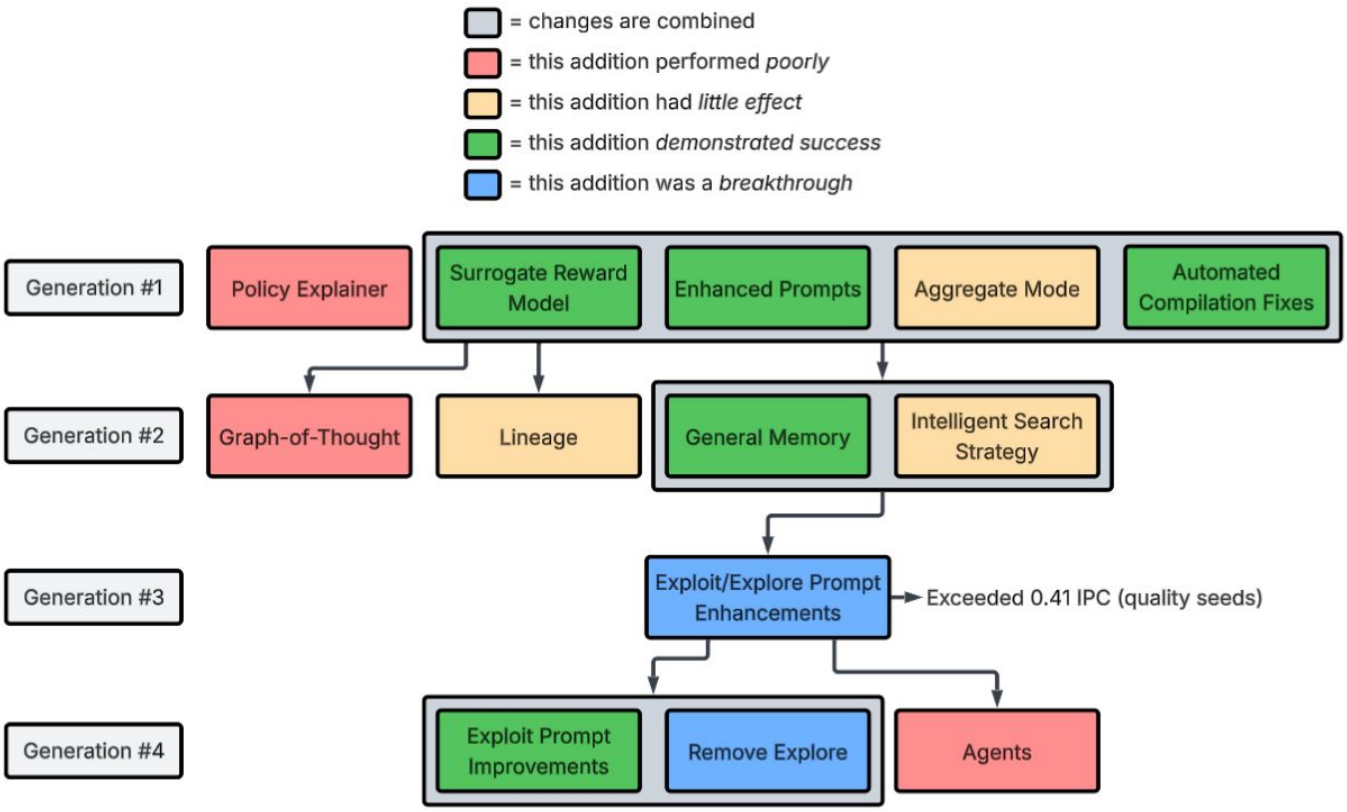

Fig. 2: Evolution of CacheForge Codebase

learning target. Additionally, different parents might have very different IPCs, but the change in IPC from a small mutation tends to be small. This allows the model to focus on the more local effect of mutations rather than trying to learn about the wider performance landscape.

In the CacheForge search loop, the surrogate reward model predicts ΔIPC for some N number of candidates specified in our loop and sorts them by their predictions. Then, it only simulates the top K, which we kept as 2, policies. It then updates our DB and the model's memory JSON to retrain the model for the next iteration. The sole purpose of the model is to eliminate the candidates with the least promise in order to speed up the iteration process.

4.4 Aggregate

While exploration and exploitation of policies both serve important roles in Cacheforge, aggregating two top performing policies can combine the strengths and mitigate the limitations of each and potentially produce an even stronger hybrid design. Although our prompting strategy for policy aggregation evolved throughout the project, the underlying idea was kept the same. Two policies would be provided to the LLM and it was instructed to fuse their strongest mechanisms into one single policy. While this approach sounds feasible, one primary aspect causes the LLM to hallucinate. Asking an LLM to combine two policies and fuse their strongest mechanisms together is a complex task and the model may invent structures or behaviors that are not present in either parent. To counter this, we provided fusion strategies that specified which mechanism should be taken from which parent and how these components should be fused. For instance, we provided the LLM with instructions to use the first parent's predictor (brain) to override the second parent's insertion decision (brawn). These targeted fusion directives served two purposes: to increase the likelihood of a higher performing aggregated policy being output, and to reduce hallucinations by

constraining the LLM to specific fusion directives, thus limiting room for hallucinations.

4.5 Lineage

To build on both the quality and continuity of our search loop so far, we added a hybrid memory module. While the surrogate reward model helps make search efficient by filtering out weak candidates, the memory system improves the *direction* of search by recalling which mutations worked well and which performed poorly. In our implementation of this module, our dataclass *Episode* stores the following information about a policy:

- Iteration number
- Policy name and natural language description
- Mutation description
- Average IPC and hit-rate outcomes per workload
- Whether the policy compiled successfully
- Source code hash and path for reproducibility
- Lineage ID (which branch the policy belongs to)

This memory system uses a 1-to-1 assignment for a lineage block to an exploit phase. The LLM is explicitly prompted with a JSON lineage block that contains all the recent episodes in the current exploit branch, including information such as metrics, descriptions, and identifiers. Rather than hard-coding a parent selection rule such as mutating only the last two policies, we ask the model to look over all episodes in the lineage to decide which prior episodes it wants to pull inspiration from and treat as the next parents. For every generated policy, the LLM is required to include a field listing the parent ids in its JSON output.

We designed the parent selection process this way in hopes of encouraging more novel and composite policies. The idea was that, by letting the LLM choose parents itself, it could find hidden gems in combining potentially strong and weak policies together to eliminate weaknesses or improve strengths. Since this scenario is less intuitive, we thought it could allow novelties to emerge.

4.6 Graph-of-Thought

One of our earlier experiments implemented the Graph of Thought (GoT) reasoning framework into CacheForge. The goal was to evolve a network of interconnected policies, or episodes, to keep track of different branches of policy information. This idea was adapted from the paper *Graph of Thoughts: Solving Elaborate Problems with Large Language Models by Besta et al.* [8] to structure the reasoning of CacheForge over policies as a graph rather than a single linear chain. This linear model limited the ability of the LLM to reason over multiple branches of policies to revisit earlier solutions that seemed promising. The implementation redesigned the linear pipeline we originally had, so policies formed a directed graph. Each candidate stored its lineage and was evaluated based on the parent nodes.

To start off, we worked to track the lineage id and parent ids of the policies. This allowed each policy to record which earlier policies it took inspiration from. We then computed a GoT-style frontier that collects all of the IDs that appear as parents and returns the remaining nodes as leaves. These remaining nodes are sorted by descending IPC, so the best branches are explored first. This mirrors the paper's ideas of ranking the most relevant thoughts and selecting promising thoughts from a larger graph. After each iteration, the structure is written into a textual graph file which lists the episodes and the parent $\rightarrow$ child edges for inspection.

Building on top of this, the storage and memory addition provide a GoT-style directed graph over all of the policies generated. It reads the JSON memory generated, converts each of the recorded episodes into a node in the graph. Each of these nodes are labeled with their respective IPC, generation, description, and mutation type. The nodes then infer their edges from the parent ids to get their parent policy. Different amounts of parents determine the type of edges. Single parents will create mutation edges and multiple parents become crossover edges. The policies and current exploration frontier modeled after GoT are retrieved and implement the GoT selection. This selection chooses the most relevant thoughts, or the highest scoring thoughts based on the evaluator to allow for further mutation. If the graph is empty, there is a fallback to a previous database.

A GoT-style evolution tree with coloring the nodes by IPC with this visualization of the structure helps make it more concrete and easier to read.

The branches, crossovers, and lineages are seen as a colored evolution tree and it makes it easier to pick out higher performance lineages. Tying it all together, we changed the prompting to build a GoT style prompt. This new prompt uses several of the frontier episodes as parallel branches for the Graph of Thought implementation. These frontier episodes are fed to the LLM to allow it to compare and reason for improvements. With this prompt and the lineage memory, the GoT implementation is complete with a graph-like exploration and policy evolution. This aligns with the paper as it works to replace regular linear reasoning with a network of paths and graph based-thinking.

4.7 Prompting Techniques

4.7.1 Exploit Strategies

To surpass the established IPC baseline of 0.413, we shifted our optimization strategy from broad exploration to targeted exploitation. This phase focuses on refining successful architectural patterns and breaking out of local maxima by enforcing strict constraints and using metric-aware prompting.

Phase I: Metric-Aware Prompt Shaping

The initial exploitation phase utilizes a specialized prompt designed to realign the LLM's objective function. Standard LLM training often conflates cache performance with hit rate, but our analysis showed that high-IPC policies required aggressive filtering (bypassing) rather than simple retention. To correct this, we deployed a prompt that emphasizes *"Maximize IPC through Smart Eviction."* This prompt introduces three critical architectural directives:

- **Objective Re-alignment:** The prompt explicitly instructs the model to prioritize IPC over Hit Rate, encouraging the eviction of "cold, non-reusing" lines even if it marginally lowered hit counts.
- **Weighted Eviction Scoring:** We guided the LLM to implement hybrid scoring formulas to balance recency and frequency. The prompt suggested a tunable weighted formula for victim selection:

$$\text{Score} = (\text{LRU_Rank} \times W) - \text{Freq} \quad (2)$$

  Here, W acts as a tunable multiplier (suggested values of 1 or 3) to weight the importance of recency against frequency.
- **Prefetch Punishment:** Contrary to standard "protection" logic, this prompt directed the model to insert prefetched lines with "stale" ranks or zero frequency, ensuring they did not pollute the working set unless validated by a demand hit.

Outcome: This phase successfully evolved our policy from an initial team best of 0.4085 to 0.41088. However, the policies began to converge at a local maximum, unable to breach the 0.4130 baseline due to the model confounding multiple variables in single mutations.

Phase II: Constraint-Based Mutation

To break the 0.41088 ceiling, we shifted from broad objective-based prompting to a strict *Constraint-Based Prompting* strategy. The primary failure mode in Phase I was the LLM attempting too many changes simultaneously (e.g., modifying eviction logic, decay rates, and bypass thresholds in a single step), which confounded the results and frequently violated the hardware budget.

Phase II imposed a rigorous **"Choose Exactly One"** constraint, forcing the LLM to perform isolated ablations. The prompt structure was engineered to enforce hardware realism and code safety through the following directives:

- **Strict Mutation Scope:** The prompt restricted changes to **5–20 lines** of code within existing functions, explicitly banning refactors, renames, or rewrites. This preserved the control flow and ensured that improvements were due to algorithmic logic rather than code structure.
- **Hardware-Realistic Memory Rules:** To guarantee strict adherence to the 64 KiB budget, the prompt prohibited dynamic allocation. It mandated that any new state must be defined as **static arrays** (8-bit or 16-bit) or lightweight **per-set hints** (≤ 1 byte per set).
- **Hallucination & Safety Guards:** Explicit instructions were included to prevent "hallucinations" of non-existent ChampSim fields. The prompt enforced core safety checks (e.g., bounds checking for `way` and `set` indices) to prevent segmentation faults during simulation.

- **Curated Mutation Menu:** We provided a specific menu of *High-Impact, Low-Cost* mutation types designed to capture behavioral hints. The LLM was required to select **exactly one** of the following:
  - **Adaptive Spatial Boosting:** Increase promotion scores when repeated < 64B deltas are observed.
  - **Dynamic Tier Assignment:** Promote to Tier-1 only when confidence is high (derived from local locality).
  - **Lightweight Behavioral Hints:** Use 2–4 bits per set to detect streaming vs. reuse patterns.
  - **Adaptive Decay Pressure:** Modulate frequency decay aggressiveness based on recent reuse patterns.
  - **Score Shaping:** Modify the $(LRU - freq_cost)$ scoring to non-linearly differentiate hot/cold lines.

Rationale: By forcing the model to select exactly one micro-architectural feature, we isolated specific gains and prevented beneficial changes from being masked by destructive ones. This "surgical" approach was critical in generating the *Adaptive Decay* and *Burst Promotion* policies that ultimately surpassed the baseline.

4.7.2 Micro-Architecture Context

One prompting technique we tried was adding micro-architecture context. The LLM has no deep knowledge of the four different types of accesses it is calculating hit-rates for. These accesses are Load, RFO, Prefetch, and Writeback, and they are not created equal. Since Load and RFO are more critical to program execution, we added a context dump to all prompts in the evolutionary loop that tells the LLM to focus on maximizing the hit-rate for Load and RFO. Looking back at Table I, we can observe that the best performing baseline, Hawkeye, demonstrated good hit-rates for *both* Load and RFO, while other baselines with worse IPC scores often only showed higher hit-rates for one of the two. Opening up the LLM to knowledge of the system's micro-architecture allows it to design an eviction method in the replacement policy to prioritize these types.

4.7.3 Seeding the LLM

Another prompting technique that we tried was seeding the LLM. This is basically a context dump for the baseline policies. Before asking the LLM to suggest improvements, we first gave it short descriptions of the baseline policies. Simple approaches included least-recently-used (LRU) and random eviction. More advanced policies used small counters, signatures, or predictors for replacement decisions.

The LLM was prompted to use ideas from these baseline policies or combine them in new ways to come up with new strategies, but it was required to keep all policies within hardware constraints. By seeding the model with these known baselines, we gave it a clear performance reference point and a solid set of mechanisms to build from. The goal of this technique was to help focus the model on generating practical improvements from the baseline policies, rather than creating unrealistic or poor designs.

4.7.4 Strict C++ Instructions

Another prompting technique we tried was giving the LLM strict rules for how the replacement policy had to be written to make sure it produces usable code. We tried this technique in hopes of reducing the amount of garbage policies that fail due to compiler errors.

We gave the LLM multiple requirements. The first was that generated code needs to be valid C++ and must compile in the context of ChampSim. The LLM was only allowed to modify the three replacement policy functions:

- `init`
- `update_replacement_state`
- `get_replacement_index`

The LLM could only use basic ways of sorting information that were set ahead of time. It was not allowed to create or manage memory dynamically. We required the logic to be easy to follow and suitable for the hardware requirements in hopes that having simple logic and avoiding features like recursion or deeply nested branches would reduce compiler errors. Additionally, the LLM had to clearly track how much metadata was being used so that the we would stay within the storage budget

requirement.

4.7.5 Exploit's Refine vs. Replace

This prompting technique tells the LLM how to mutate the current cache replacement policy. To accomplish this, We used two modes based on whether we want small improvements or an entirely new policy.

The first mode was refinement mode. Refinement mode is used for small, specific improvements to an existing policy. The LLM is asked to change only minor parts of the policy such as thresholds, counter sizes, update rules, or initialization behavior. Changing the overall structure or metadata structure are disallowed. All changes must stay within the remaining metadata budget, and for each refinement, the LLM is required to provide:

1) A short explanation of the change.
2) A minimal code differentiation showing only the lines that were modified.
3) A brief description of the expected behavior.

This mode is primarily used during later stages of tuning, and the goal of this was to boost pre-existing policy.

The second mode was replacement mode. Replacement mode is used to design a completely new policy from scratch. In replacement mode, the LLM is permitted to change the full structure of the policy and introduce new ideas, as long as the total metadata stays within the predetermined fixed storage budget. For each replacement, the LLM must provide:

1) A high-level description of how the policy works.
2) A full C++ implementation of the modified files.
3) A calculation of the total metadata size.
4) A short comparison to the previous policy.

This mode is used during early exploration or when performance improvements stall. This provided a way for the LLM to "escape" the direction a design was going.

4.8 Agents

As we made progress, our prompts and ideas became increasingly complex. It became clear that relying on a single LLM call for end-to-end policy generation played a role in degrading performance. The LLM frequently produced policies with compile errors, inconsistencies, or incomplete mechanisms. To address this, we implemented a multi-agent framework in which 5 specialized agents were introduced, each responsible for a distinct stage of policy generation.

The architect agent is a cache replacement policy expert instructed to output strict design information for four potential future policy implementations. The advisor agent is an expert inspector of policy design ideas, primarily focused on identifying flaws. The advisor is responsible for inspecting the four policy designs produced by the architect and selecting the best two for development. The developer agent is not a cache replacement expert, but instead is optimized to develop code based on the architect's design. The debugger agent inspects the policies generated by the developer to check for simple and common compiler errors. Lastly, the refiner is an expert inspector finding crucial IPC-limiting flaws in policy implementations.

This architecture was implemented from two different starting seeds Hawkeye then SHiP++, with and without memory, with and without micro-architecture context, and both not refined and refined. The primary changes in our refined agent architecture shifted more responsibility to the architect. This meant that the architecture was required to produce more detailed output and structured blueprints with the goal of further reducing the load on the developer agent.

5 Results

5.1 Policy Explainer

Our policy explainer was tested using the same codebase both with and without the policy explainer. As shown in Figure 3, the inclusion of the policy explainer resulted in lower performing policies, as the policy distribution showed a higher variation and a lower median. Overall, these results suggest that the policy explainer had no positive impact on the quality of generated policies.

5.2 Surrogate Reward Model

Our surrogate reward model focused mainly on pruning potentially garbage or low performance

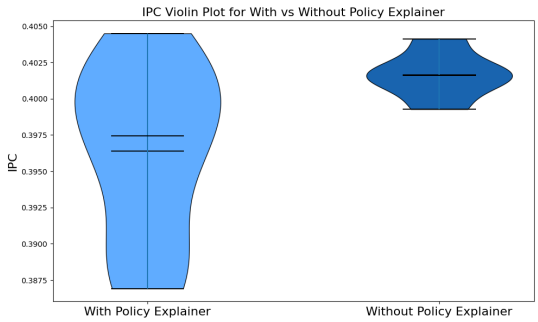

Fig. 3: Results from with and without Policy Explainer

policies before they are simulated to speed up the iteration process. The result was only one low outlier policy that performed at around 0.39 IPC and a tighter, less left-skewed distribution around 0.42 for the remainder of policies (see Figure 4).

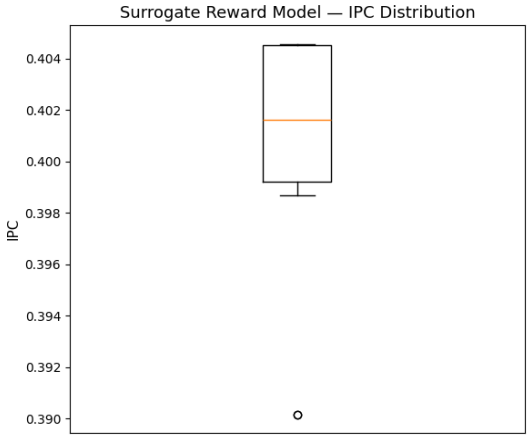

Fig. 4: Results from Surrogate Reward Model

5.3 Aggregate

The fusion of two novel cache replacement policies could, in theory, result in new novel and high performing policies. However, our results indicate the opposite. When the LLM was instructed to exploit another policy, the resulting policies significantly outperformed those generated through aggregation. Ignoring outliers, the minimum harmonic mean IPC for policies produced by exploitation exceeded the beginning of the first quartile for policies produced from aggregation, the first quartile for exploitation exceeded the median of aggregation, and the median of exploitation exceeded the max of aggregation (see Figure 5).

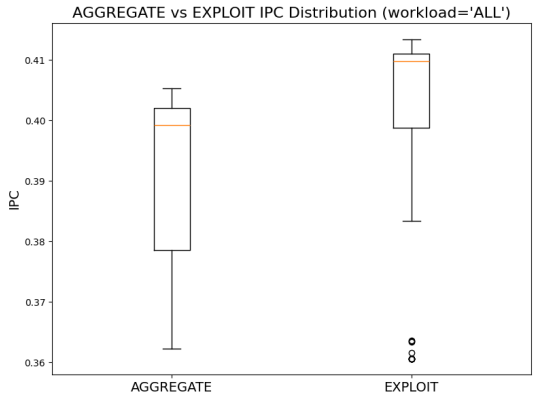

Fig. 5: Results from Exploitation vs Aggregation on best version of code

5.4 Lineage

Our approach that allows the LLM to self-analyze its exploit phase lineage gave somewhat promising results. Several policies exceeded an IPC of 0.407 and over 75% of policies exceeded the 0.4 threshold (see Figure 6).

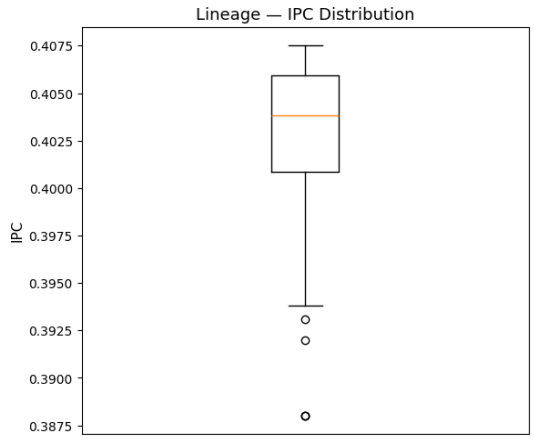

Fig. 6: Results from LLM Self-Analysis of Lineage

5.5 Prompting Techniques

5.5.1 Exploit Strategies

Figure 7 shows the IPC distributions for both Phase I and Phase II of exploitation, which was evaluated across all workloads.

Phase I shows a moderate improvement in central tendency, with most policies clustering slightly above .40 IPC. However, the distribution is relatively wide, indicating that while some metric-aware exploitation attempts were effective, performance

remained relatively inconsistent across generated policies. Even though the performance was inconsistent, Phase I was able to produce a temporary team best of 0.40856 and then another temporary team best of $.41088$, before it eventually plateaued.

Phase II demonstrates a clear upward shift in performance and improved stability. The median IPC increases similarly to Phase I, but the interquartile range noticeably shrinks. While there were a small number of low outliers, the majority of Phase II policies consistently clustered near the upper end of the observed IPC range. This concentration indicates an improved peak performance and a reduced variance across generated policies. Phase II was able to increase our previous team best of $.41088$ to our current team best of $.41366585$.

Overall, the results show that Phase I was able to produce policies with high IPC's, but it was ultimately Phase II that pushed us over the baseline. Ultimately, Phase II's exploitation producing more reliable and higher-performing policies than Phase I indicates that tightly constrained, isolated mutations were more effective than broader, metric-guided refinements.

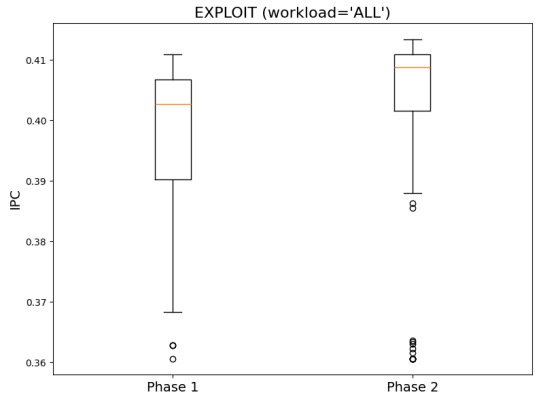

Fig. 7: Results from Phase I and Phase II of Exploit Prompting

5.5.2 *Micro-Architecture Context*

Adding micro-architecture context to the prompt did not lead to strong performance gains. In fact, the median IPC is much lower than several other prompting techniques, indicating that adding the context alone did not consistently improve performance. However, policies where the LLM was able to increase storage typically had a higher IPC than policies that took up less storage. Considering the fact that several policies reached higher IPC upper bounds, this indicates that the micro-architecture prompting can be beneficial when effectively leveraged.

Overall, the wide spread in performance suggests the LLM struggled to consistently convert architectural knowledge into effective eviction strategies (see Figure 8).

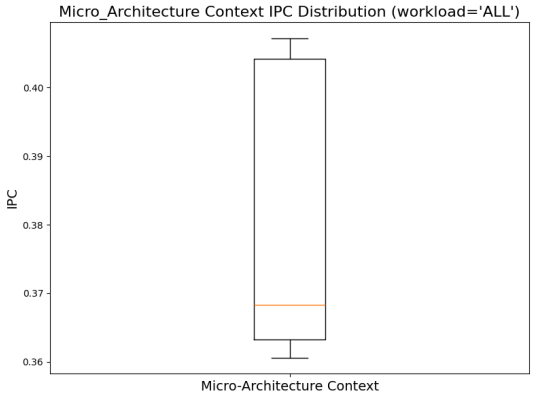

Fig. 8: Results from adding Micro-Architecture context to the prompt

5.5.3 *Seeding the LLM*

Seeding the LLM with the baseline cache replacement policies did not lead to strong performance gains. Although the generated policies were generally well-structured and clearly based on known replacement strategies, this alone was not enough to consistently improve IPC. The median IPC for policies created by seeding the LLM was relatively low compared to other policies we were generating and significantly lower than the IPC of the baseline policies (see Figure 9).

Seeding the LLM proved useful for guiding the model toward valid and stable designs, but it did not lead to strong performance improvements alone. This highlights an important limitation of relying only on baseline context to drive optimization.

5.5.4 *Strict C++ Instructions*

Prompting the LLM with strict C++ instructions shows consistent performance across all generated policies around approximately 0.398 IPC (see Figure 10). The low variance suggests that the strict constraints of the C++ instructions narrowed the general behavior of policies. Considering the fact that there are no strong high-performing outliers, it

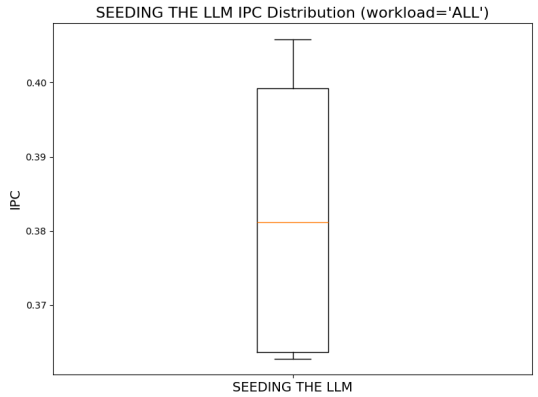

Fig. 9: Results from Seeding the LLM by context dumping different existing baseline policies (Hawkeye, SHiP++, LFRU, etc.)

is possible that the strict rules limited the LLM's exploration into more aggressive optimization strategies. Despite no IPC gain, there were significantly less compiler errors during experimentation, which led to more successful iterations.

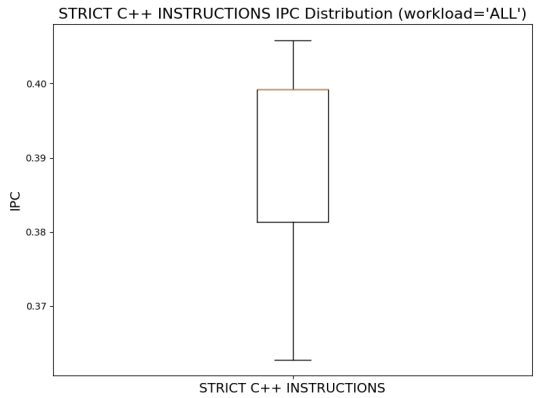

Fig. 10: Results from giving direct C++ Instructions

5.5.5 *Exploit's Refine vs. Replace*

Exploit's refinement and replacement modes were effective. They yielded a median IPC of slightly above .40 (see Figure 11). The narrow inter-quartile range suggest that both refinement and replacement modes produced consistently high-performing policies. This prompting technique's upper range reached the second highest IPC values observed in all of the techniques, showing that this approach is certainly effective at producing high quality policies.

Refinement mode is best for late-stage performance tuning, allowing small changes to extract additional gains from existing policies. Replacement mode provides an effective mechanism for exiting a design's "train-of-thought", allowing exploration of a fundamentally different policy when progress stalls.

Combining refinement and replacement offers a strong balance between exploitation of existing designs and exploration of new strategies, resulting in reliable performance improvements.

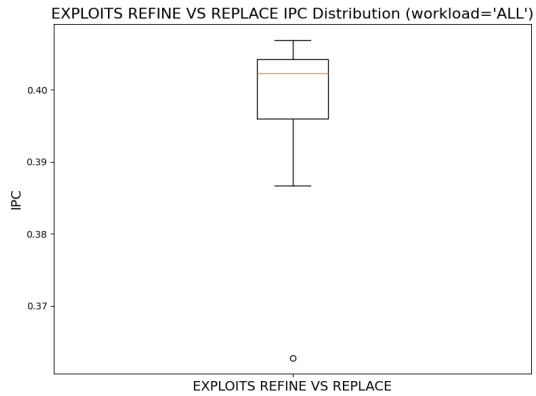

Fig. 11: Results from Exploits Refinement vs. Replacement modes

5.6 Graph-of-Thought

Across different runs, Graph of Thought was never able to discover a policy that outperformed all of the baselines in terms of IPC. Figure 12 shows all of the successfully compiled policies around 0.39-0.404 IPC, with the best runs matching each other, but never surpassing the Hawkeye baseline. During these runs, the GoT loop repeatedly struggled with compiler errors, such as lineage frontiers being empty and `compile_ok` remaining false for promising thought candidates like RRIP with Predictor. There were also issues with the InitReplacementState and UpdateReplacementState, which after two failed attempts, these policies were forced to be skipped, leading to starvation of GoT. Overall, the Graph of Thought did small works around the baseline policies, but did not reach the necessary performance.

5.7 Agents

The policies generated by agents performed below expectations, with the median harmonic mean IPC typically well below 0.40 (see Figure 13).

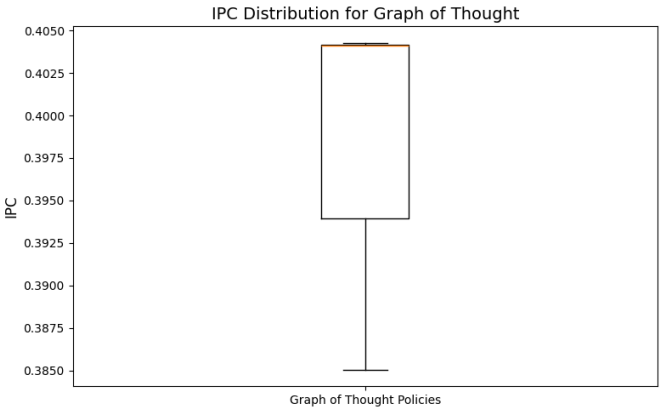

Fig. 12: Results from Graph of Thought

Our agents experiments were run across two different seeds, Hawkeye and SHiP++. The results from this study indicate that SHiP++ consistently serves as the better starting seed, producing more policies with higher harmonic mean IPC (see Figure 14). This suggests that Hawkeye may be more difficult for an LLM to meaningfully exploit or refine compared to SHiP++.

6 Novel Policies

6.1 Analysis of Novel High-Performance Policies

Our evolutionary search using `gpt-oss:20b` yielded five distinct policies that successfully surpassed the hard baseline of 0.413081 (Hawkeye). Unlike the baseline LFRU which relies solely on frequency and recency, the top-performing policies universally converged on a single architectural theme: **Spatial-Aware Frequency Modulation**. By integrating lightweight, per-set spatial signals, these policies effectively decouple "protection" from "recency," allowing them to filter pollution more aggressively than traditional replacement strategies.

Common Architectural Themes

The success of these top five policies can be attributed to three key deviations from standard LFRU:

1) **Spatial-Aware Decay:** Instead of decaying all lines uniformly, these policies use per-set "spatial hints" (detecting small address deltas) to protect lines that are part of a streaming pattern while aggressively decaying random accesses.
2) **Burst Promotion:** Workloads like `omnetpp` and `mcf` exhibit "reuse streaks" where data is used heavily for a short burst and then abandoned. Standard policies fail here because the line is evicted before its frequency counter ramps up. The generated policies introduce "Burst Promotion" to immediately jump-start frequency counts when a streak is detected.
3) **Granular Adaptivity:** Unlike Hawkeye, which relies on a global PC-based predictor, these policies utilize per-set counters (e.g., `spatial_hint`, `reuse_streak`). This allows for finer-grained adaptation to transient phases within specific cache sets.

Detailed Policy Analysis

Below is an analysis of the mutations that defined our top-performing candidates:

- **Adaptive Decay Pressure (IPC: 0.413666)**
  textitMutation Strategy: This policy introduces a dynamic decay rate. It utilizes a per-set `spatial_hint` bit that is set when access deltas are < 64 bytes.
  - **Mechanism:** When spatial locality is detected, the frequency decay logic is altered to prevent lines from dropping to zero (the eviction threshold). Conversely, sets without spatial locality face $2\times$ faster decay pressure.
  - **Why it wins:** It solves the "Thrashing" problem. In high-contention sets, it prioritizes lines that are part of a spatial stream, preserving the working set while rapidly evicting random noise.
- **Burst-Promotion via Short Reuse Streaks (IPC: 0.413666)**
  Mutation Strategy: This policy adds a 1-byte counter per set to track consecutive spatial hits.
  - **Mechanism:** If the `reuse_streak` counter reaches 2, the *next* inserted line is immediately promoted to maximum frequency (7), bypassing the slow "ramp-up" phase of standard LFRU.
  - **Why it wins:** It effectively targets "Scan" patterns common in memory-intensive workloads. By predicting that a streak

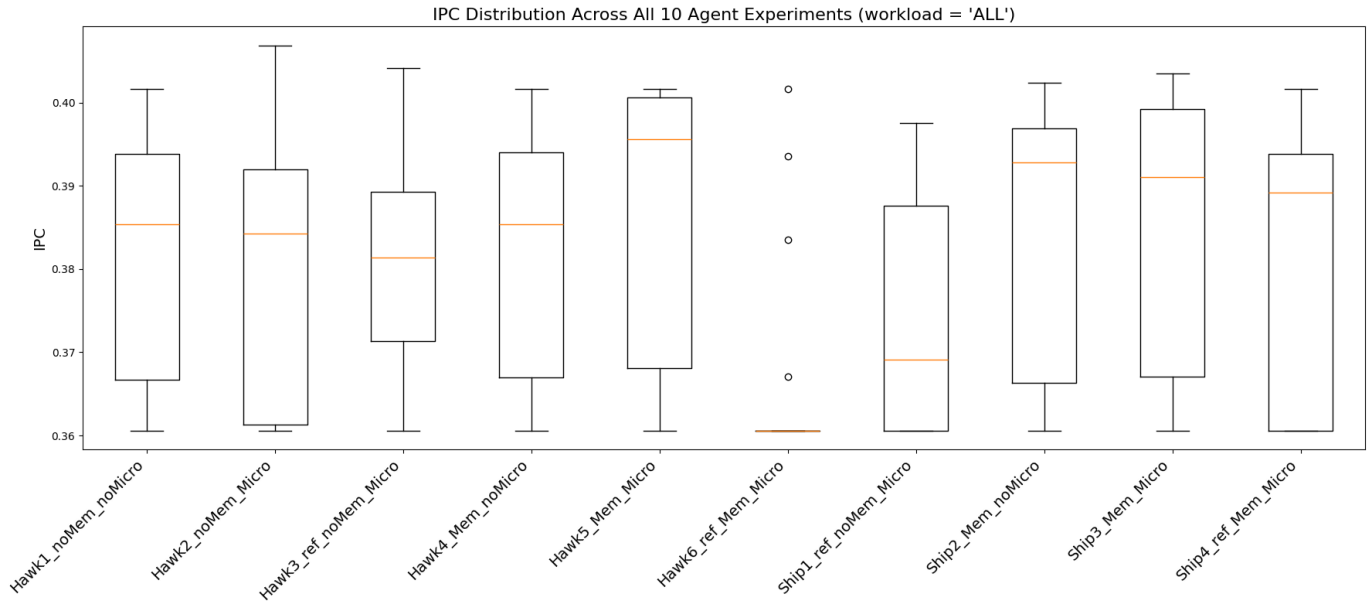

Fig. 13: Results from every experiment run with the agent architecture

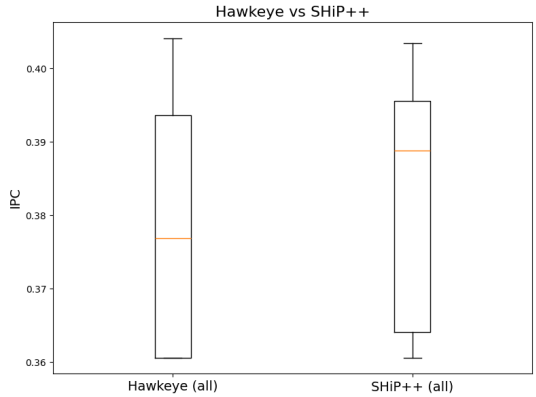

Fig. 14: Agent experiments with aggregated Hawkeye results (removing first two hawkeye experiments to match the SHiP++ experiments) vs aggregated SHiP++ results

implies immediate future reuse, it protects lines that would otherwise be evicted by LRU-based policies before they could prove their utility.

- **Adaptive Decay with Streaming Hint (IPC: 0.413659)**
  Mutation Strategy: An evolution of the Adaptive Decay policy, this variant adds a "pollution filter" for streaming data.
  - **Mechanism:** It detects long, continuous streams via a `small_delta_count`. If a set is identified as "streaming" (count ≥ 2), it penalizes the insertion Tier and dampens frequency growth.
  - **Why it wins:** It differentiates between "useful reuse" and "pollution." While it protects spatial locality (like Adaptive Decay), it stops one-off streaming data from occupying high-priority Tier-1 slots.
- **Conditional Tiering (IPC: 0.413658)**
  Mutation Strategy: This policy fundamentally changes the insertion logic.
  - **Mechanism:** Instead of always inserting new lines into Tier-1 (protected), it performs a conditional check. A line is only promoted to Tier-1 if it has a frequency ≥ 2 or belongs to a set with active spatial hints.
  - **Why it wins:** It reduces the "Dead on Arrival" cost. New lines must "prove" their worth (via reuse or spatial association) before displacing existing protected content, significantly increasing IPC on mixed workloads.

Storage Overhead Analysis

A critical requirement of the competition was adhering to a strict 64 KiB budget. These policies

are remarkably efficient because they rely on per-set metadata rather than per-line state.**Standard LFRU State:**

- 1 byte per line (Frequency + LRU + Tier) $\times$ 32,768 lines = **32 KiB**.

Novel Policy Overhead (e.g., Adaptive Decay):

- `spatial_hint`: 1 byte per set $\times$ 2048 sets = 2 KiB.
- `set_counter`: 1 byte per set $\times$ 2048 sets = 2 KiB.
- `last_addr`: 8 bytes per set $\times$ 2048 sets = 16 KiB.

Total Storage: 32 (Base) $+ 2 + 2 + 16 =$ **52 KiB**. All generated policies consume between **52 KB** and **54 KB**, leaving nearly 10 KB of headroom while outperforming the complex Hawkeye policy.

7 Discussion

7.1 Policy Explainer

Our goal in implementing a policy explainer was to enhance LLM memory, consistency, and reliability. However, our results suggest that the policy explainer had no positive correlation in improving the quality of generated policies. One potential cause for this is hallucination from the LLM in its explanations of required fields. One LLM call was responsible for the entire policy generation process, which included the explanation of the policy. Given the consistent usage of models with low parameter count, for example GPT's 20B parameter model, the LLM is more prone to these hallucinations. Furthermore, if these fields are hallucinated, the LLM will later receive the lineage_block as input and be instructed to inspect the incorrect or fabricated information that was generated by these hallucinations.

7.2 Surrogate Reward Model

From our results, it seems the surrogate reward model accomplished its task to some extent. We observed that the surrogate reward model, while its design technically plays no part in improving the IPC of top-performing policies, does have an impact in skipping over potentially low-performance policies. The lighter left-skew and mere one outlier shows that the distribution is more balanced, unlike a majority of distributions from CacheForge that often have a heavy left-skew.

One issue we ran into with the surrogate reward model was its architecture. Initially, we used an MLP Regressor, a type of artificial neural network, to predict ΔIPC of new policy candidates. These predictions proved to be wildly inaccurate, as a neural network tends to overfit with the little data we were generating in early iterations. Once we switched our model to an SGD Regressor, a linear regression model, our ΔIPC predictions were far more accurate, as a linear regression is less prone to overfitting with such little and simple data. That is the model we used to generate the results you see in Figure 4.

7.3 Aggregate

The goal of adding aggregate mode was to fuse two strong policies into a single, stronger policy by combining the strong qualities and mitigating the limitations of the two original policies. While this idea is intelligent at a first glance, our results suggest otherwise as the aggregate step significantly underperformed in comparison to exploit mode. This is likely caused by LLM hallucinations and thus weaker policies when provided with more vague instructions. In contrast, the LLM excelled when provided a more detailed and strict set of instructions in exploit mode.

7.4 Lineage

Our approach to lineage allowed the LLM to self-analyze the lineage of its entire exploit phase, prompting it to take inspiration how it saw fit instead of programming a fixed parent selection process. While this implementation was intended to enable the generation of more novel policies, it instead caused frequent hallucination of non-existent parent policy IDs by the weak, 20B parameter open-source model we used for our experiments.

Despite these hallucinations, the model was still able to produce policies that exceeded an IPC of 0.407, which was a few tenths below our previously best-performing policies. We decided to not continue with this lineage implementation, however, due to the sheer amount of wasted LLM calls and runtime that was taken up by ID hallucinations. Additionally, the resultant policies performed

moderately but were not producing novel heuristic/baseline combinations. This idea certainly had potential, but other codebase changes in its generation as noted in Figure 2 performed better, so we labeled it as having little effect in improving our overall system.

7.5 Graph-of-Thought

Graph of Thought was not able to produce consistent compilable policies or higher IPC than the baselines. With the limitations discussed in Section 5.6, we shifted our focus away from the implementation. While the issues from the results were addressed in later experiments and were able to be fixed, we ultimately decided to go into a different direction with exploring prompting techniques instead.

7.6 Agents

Agents were introduced to reduce the weight upon one single LLM call handling end-to-end policy generation. However, the results were significantly worse compared to our flagship version of CacheForge. Several factors likely contributed to this, particularly the quality of the prompts and increased number of steps where information could be lost or distorted.

In our best version of CacheForge, we ignore exploration and strictly exploit the highest performing policies, providing strict instructions on how to do so. In contrast, our agent architecture still includes the exploration step and also gives the LLM more freedom upon policy generation. This may have encouraged the LLM to hallucinate qualities and mutations that do not increase harmonic mean IPC. Additionally, introducing agents requires more LLM calls, potentially increasing the complexity of the policy generation process. Rather than smoothly distributing responsibilities across agents, this simply introduced more opportunities for information loss or misinterpretation. Important information from the architect may have been lost or misinterpreted when it reached the developer, the developed code may have been misunderstood by the refiner, etc.

Our implemented agent infrastructure did not produce high quality policies. However, this does not mean that using agents cannot help. With the same high-quality prompts from our highest performing branch, along with restructuring the architecture of the agents, agents could prove effective. One potential improvement could be redesigning the developer agent to be an expert on cache replacement, providing it more context regarding the code it is implementing. This would increase the workload of the developer agent, but it has the potential to increase policy quality.

7.7 Evolutionary Loop: Exploration vs. Exploitation

The architecture of our evolutionary loop underwent significant refinement throughout the competition. Our simple loop evolved into a targeted, iterative refinement engine capable of maximizing the limited reasoning capabilities of the `gpt-oss:20b` model.

Iteration 1: The Baseline Framework
Our initial framework relied on a simple two-state cycle: *Explore* and *Exploit*. The primary function of the Explore stage was to generate diverse starting seeds, which the Exploit stage would then attempt to optimize. However, without specific guidance, the model struggled to generate competitive policies. Initial runs yielded harmonic mean IPC (hm_IPC) values hovering around **0.40**, significantly trailing the **Hawkeye** baseline (0.413). This demonstrated that blind exploration was insufficient for this optimization landscape.

Iteration 2: Targeted Prompting & Aggregation
To improve candidate quality, we introduced an *Aggregate* stage and refined our prompting strategies to be more domain-specific. The goal was to transform the search from a ”needle in a haystack” probability game into a directed search. This iteration produced our first significant jump in performance; during a specific run utilizing targeted ”Smart Eviction” prompts, we achieved two policies in the upper **0.410** range. While this was a marked improvement over the 0.40 starting point, it still failed to breach the Hawkeye baseline.

Iteration 3: The Architect-Developer Split
Inspired by the *AutoAgents* and *ReAct* frameworks, we restructured the loop to reduce the cognitive load on the 20B model. We hypothesized that asking the

model to both *ideate* and *implement* simultaneously was causing performance degradation. We split the process into two distinct roles: an *Architect* (strategy generation) and a *Developer* (implementation). While this approach improved the syntactic correctness of the Exploit stage, the Explore stage remained a bottleneck. We encountered a "garbage in, garbage out" cycle: if the Architect generated fundamentally flawed starting points, the Developer could not exploit them into high-performance policies. This confirmed that you cannot exploit a fundamentally bad policy.

Iteration 4: Direct Exploitation (Breakthrough) The final and most successful iteration emerged from the realization that "exploration" was unnecessary when high-quality seeds already existed. We re-engineered the loop to strictly exploit the top-performing policies (specifically the 0.4108 candidates from Iteration 2) using a strict exploit strategy:

- **Seed Selection:** The loop iterated exclusively through the Top 5 policies.
- **Micro-Iterations:** We limited the loop to 3 targeted exploit iterations per policy.
- **Atomic Mutation:** We enforced the "Choose Exactly One" mutation constraint.

This shift allowed us to systematically climb from 0.4108 to match and eventually surpass the 0.413081 baseline over 50 times. By using the high-performing LFRU policies as seeds, we provided the model with a stable foundation that was easier to mutate than the complex Hawkeye code.

Significance and Future Outlook
The final evolutionary loop enabled us to become the first team to beat the Hawkeye baseline using open-source models (`gpt-oss:20b`). This achievement is particularly significant given the prevailing sentiment among the class that surpassing the baseline with a 20B parameter model was "too difficult" or potentially impossible. Our results prove that success was not a matter of model size, but of prompt engineering strategy.

For future teams, we recommend bypassing broad exploration in favor of aggressive, constraint-based exploitation of the best available "known goods." We believe further potential remains in applying this "Direct Exploitation" loop to the Hawkeye baseline itself; utilizing its unused 32KB storage budget with our atomic mutation strategy could likely yield even higher IPC gains.

8 Contributions

Table II shows the contributions from each group member towards the evolution of CacheForge.

Member	CacheForge Contribution
Meseker	Surrogate Reward Model, Evolutionary Loops, and Exploit Prompt
Ethan	Agents, Lineage, Micro-Architecture, Policy Explainer
Rithik	Lineage, Micro-Architecture, Surrogate Reward Model, Agents
Madison	Graph-of-Thought and OpenAI Experiments
Savanah	Exploit Prompt, Micro-Architecture, Seeding LLM, Strict C++, Refine vs. Replace
Brantley	Policy Explainer, Debug and validate replacement policies

TABLE II: Group Member Contributions

Table III shows the contributions from each group members towards this paper.

Member	Paper Contribution
Meseker	Exploit Strategies Methodology, Novel Policies/What worked well, Evolutionary Loop Discussion
Ethan	Agents, Aggregate, Policy Explainer
Rithik	Lineage, Micro-Architecture Methodology, Surrogate Reward Model, 2.1, 2.2, 4.1
Madison	2.3, 3, Graph of Thought
Savanah	Exploit Strategies Results, Micro-Architecture Context Results, Seeding the LLM, Strict C++ Instructions, Exploit's Refine vs. Replace
Brantley	Introduction, Replacement Policy

TABLE III: Paper Contributions

9 References

[1] A. Fawzi and B. R. Paredes, "FunSearch: Making New Discoveries in Mathematical Sciences Using Large Language Models," *DeepMind Discover*, Dec. 14, 2023. [Online]. Available: https://deepmind.google/discover/blog/funsearch-making-new-discoveries-in-mathematical-sciences-using-large-language-models/

[2] J. L. Henning, "SPEC CPU2006 Benchmark Descriptions," ACM SIGARCH Computer Architecture News, vol. 34, no. 4, pp. 1–17, Sept. 2006, doi: 10.1145/1186736.1186737.

[3] R. L. Mattson, J. Gecsei, D. R. Slutz, and I. L. Traiger, ”Evaluation techniques for storage hierarchies,” IBM Systems Journal, vol. 9, no. 2, pp. 78–117, 1970, doi: 10.1147/sj.92.0078.

[4] ISCA 2026 Submission #311, CACHEFORGE: LLM-Guided End-to-End Generative Cache Replacement Policy for Performance and Hardware Efficiency,

[5] N. Gober et al., “The Championship Simulator: Architectural Simulation for Education and Competition,” arXiv.org, https://arxiv.org/abs/2210.14324 .

[6] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, ”ReAct: Synergizing Reasoning and Acting in Language Models,” in *International Conference on Learning Representations (ICLR)*, 2023. [Online]. Available: https://arxiv.org/abs/2210.03629

[7] G. Chen, S. Dong, Y. Shu, G. Zhang, J. Sesay, B. Karlsson, J. Fu, and Y. Shi, ”AutoAgents: A Framework for Automatic Agent Generation,” *arXiv preprint arXiv:2309.17288*, 2023.

[8] M. Besta et al., “Graph of thoughts: Solving elaborate problems with large language models,” Proceedings of the AAAI Conference on Artificial Intelligence, https://doi.org/10.1609/aaai.v38i16.29720 (accessed Dec. 8, 2025).